MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers Informatics and Microelectronics

Department of Software Engineering and Automation

Report on laboratory work 3

The command line shell pipes and text filters

Course Operating Systems

Done by: st. gr. FAF-242, Tacu Igor
Checked by: Reițman Patricia

Chișinău, 2026

Purpose and execution record

All twenty-four Lab 3 figures now use native Terminal screenshots supplied by the author from direct execution and source inspection on the Ubuntu server on 9 November 2026. Some screenshots cover two adjacent questions. The initial predictions and their original run are retained in predictions.json; the screenshots document subsequent verification runs. The exact text logs and source are also included in the repository. The submission was prepared with coding assistance, and MIT xv6 is identified as upstream in the source provenance file.

Part 1 Move fast and find help

Cd and echo are Bash builtins. In the interactive capture, ls is an alias for ls --color=auto; which resolves the underlying external executable to /usr/bin/ls. Cd must change the current directory of the persistent shell itself. A child process could change only its own directory. In this run PATH had nine entries and which resolved ls to /usr/bin/ls. A nonexistent command returned 127. The ls option -t sorts by modification time, newest first. Man, man -k, ls --help and help cd supplied local help.

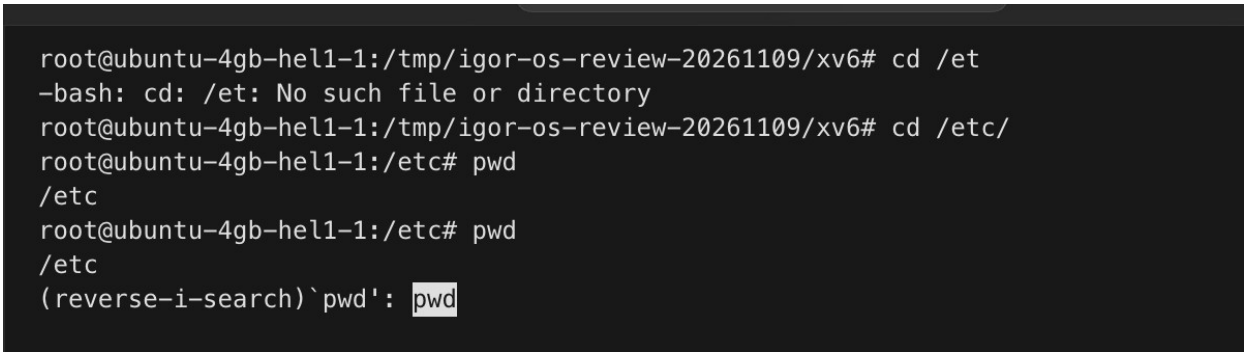


Figure 1.1 – Tab completion and reverse history search

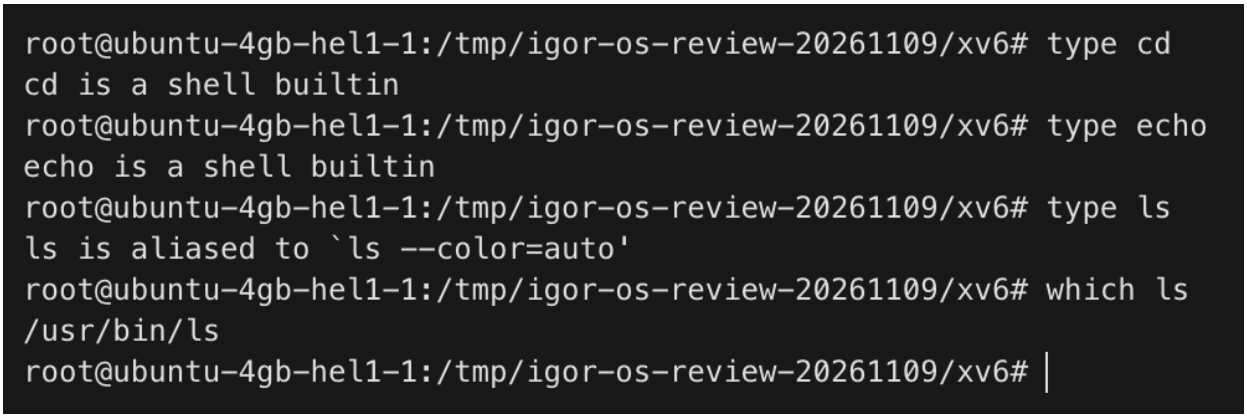


Figure 1.2 – Type and which



Figure 1.3 – Directories in PATH

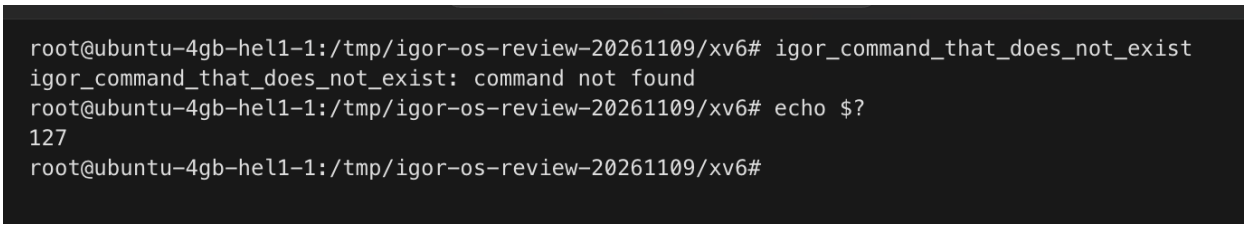


Figure 1.4 – Exit status of a missing command

Part 2 Predictions and shell expansion

The required play directory contained a.c, b.c, file1.txt, file2.txt, notes.txt and alfa beta.txt. Each prediction below was recorded before execution; timestamps and complete stderr are retained in the JSON evidence. Differences in line layout from ls are presentation differences, not additional filenames.

Command	Prediction before run	Observed output and status	Explanation
echo *.c	a.c b.c	a.c b.c status 0	Bash expands the pattern before echo starts.
echo file?.txt	file1.txt file2.txt	file1.txt file2.txt status 0	Question mark matches one character.
echo [a-b]*	a.c alfa beta.txt b.c	a.c alfa beta.txt b.c status 0	The range matches names beginning with a or b.
echo *.xyz	literal *.xyz because no match exists	*.xyz status 0	The unmatched pattern remains literal with the default settings.
ls alfa beta.txt	two missing operands and a nonzero status	ls: cannot access 'alfa': No such file or directory ls: cannot access 'beta.txt': No such file or directory status 2	Unquoted whitespace creates two operands.
ls "alfa beta.txt"	one file named alfa beta.txt	alfa beta.txt status 0	Quotes preserve one pathname containing a space.
echo '$HOME' "$HOME" \ $HOME	$HOME /root $HOME	$HOME /root $HOME status 0	Single quotes and backslash protect the dollar sign; double quotes permit expansion.
echo {1..5} ; echo $((7 * 6))	1 2 3 4 5 then 42	1 2 3 4 5 42 status 0	Brace expansion lists values; arithmetic expansion computes 42.
echo "today is $(date +%A), I am $(whoami)"	current weekday and current login identity	today is Monday, I am root status 0	Command substitution supplies the current weekday and identity.
touch log{1..3}.txt && ls log*	log1.txt, log2.txt, log3.txt	log1.txt log2.txt log3.txt status 0	Brace expansion creates three filenames.

Echo does not discover files; Bash supplies the expanded arguments. The unquoted space in ls alfa beta.txt produces two failed lookups. Both ls "alfa beta.txt" and ls alfa\ beta.txt address the single file correctly. Single quotes retain literal text; double quotes still allow parameter and command substitution. GNU ls may display quotes for a space-containing filename when its output is a terminal, but those display quotes are not stored in the name.

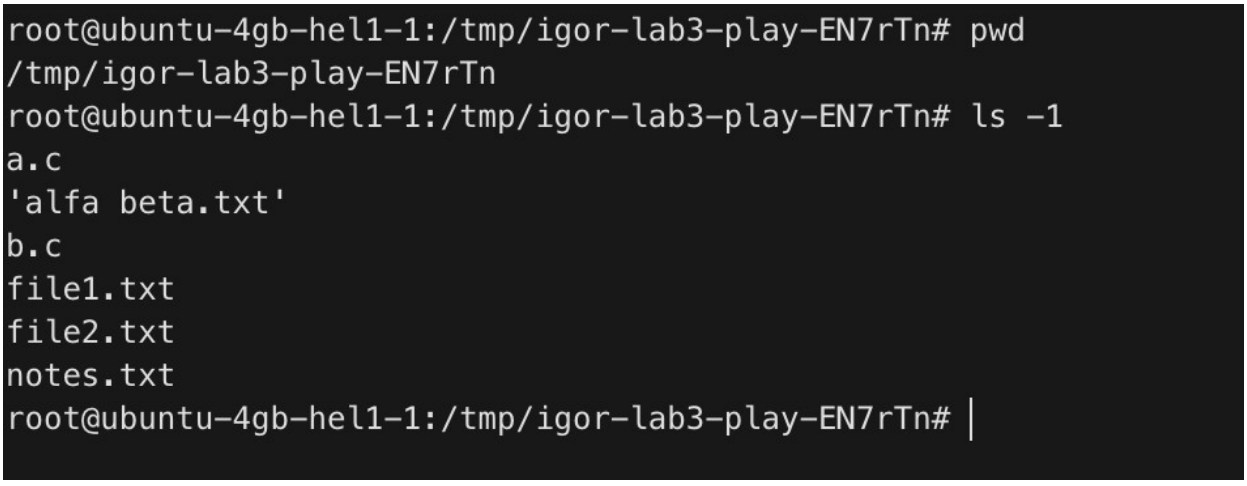


Figure 2.1 – Files in the required play directory

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo *.c
a.c b.c
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo file?.txt
file1.txt file2.txt
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo [a-b]*
a.c alfa beta.txt b.c
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo *.xyz
*.xyz
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 2.2 – Globbing results

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ls "alfa beta.txt"
'alfa beta.txt'
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ls alfa\ beta.txt
'alfa beta.txt'
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 2.3 – A space containing name and the quoted fix

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ls afla beta.txt
ls: cannot access 'afla': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ls "alfa beta.txt"
'alfa beta.txt'
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ls afla\ beta.txt
ls: cannot access 'afla beta.txt': No such file or directory
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo '$HOME' "$HOME" \$HOME
$HOME /root $HOME
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 2.4 – Single double and escaped dollar quoting

Part 3 Redirection chaining and jobs

Out.txt received stdout, while err.txt received the diagnostic for /nope. In > all.txt 2>&1, descriptor 1 is changed first and descriptor 2 duplicates that new target. In 2>&1 > all.txt, stderr keeps the original stdout target while only stdout moves to the file. Wc -l < access.log receives stdin and therefore prints a count only; with access.log as an operand it also knows and prints the filename. A semicolon sequences regardless of the previous status; AND runs the next command only after success; OR runs it only after failure. The background sleep was listed, terminated and reaped with status 143.

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ls /etc /nope > out.txt 2> err.txt
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# wc -l out.txt
216 out.txt
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# head -n 3 out.txt
/etc:
acpi
adduser.conf
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# cat err.txt
ls: cannot access '/nope': No such file or directory
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 3.1 – Stdout and stderr in separate files

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ls /etc /nope > all.txt 2>&1
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# grep '/nope' all.txt
ls: cannot access '/nope': No such file or directory
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ls /etc /nope 2>&1 > all.txt
ls: cannot access '/nope': No such file or directory
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# grep '/nope' all.txt
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo "grep status: $?"
grep status: 1
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 3.2 – Order of descriptor redirections

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# false ; echo "semicolon continues"
semicolon continues
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# false && echo "AND should not print"
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# false || echo "OR fallback"
OR fallback
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 3.3 – Semicolon AND and OR

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# sleep 100 &
[1] 3316845
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# lab3_job_pid=$!
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# jobs
[1]+  Running                 sleep 100 &
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# kill "$lab3_job_pid"
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# wait "$lab3_job_pid"
[1]+  Terminated              sleep 100
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo "job status: $?"
job status: 143
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# jobs
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 3.4 – Background job and termination

Part 4 Answers to the eight log questions

The artificial dataset uses the exact 2026-10-05 generator supplied in the handout. It is shared assignment input rather than real web traffic. All 500 lines were regenerated. The pipelines compare the result field, preserve tied maxima and sort their output. The four page counts sum to 500 and the seven user failure counts sum to 100. In Q8 the long listing has one extra total header line; the entries are otherwise the same.

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# for ((i=1; i<=500; i++)); do
  status=OK
  ((i % 5 == 0)) && status=FAIL
  printf '2026-10-05 10:%02d user%d %s /page%d\n' \
    "$((i%60))" "$((i%7))" "$status" "$((i%4))"
done > access.log
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# head -n 3 access.log
wc -l access.log
awk '$4 == "FAIL" {n++} END {print n+0}' access.log
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
100
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# awk '{print $5}' access.log | sort -u
awk '{print $5}' access.log | sort -u | wc -l
/page0
/page1
/page2
/page3
4
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 4.1–4.2 – Questions 1 and 2 results

```
root@ubuntu-4gb-hel1-1:~# cd /tmp/igor-lab3-play-EN7rTn
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo "Q3: Requests per page"
Q3: Requests per page
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# awk '{n[$5]++} END {for(p in n) print n[p],p}' access.l
og | sort -k1,1nr -k2
125 /page0
125 /page1
125 /page2
125 /page3
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo "Q4: Users with the most failures"
Q4: Users with the most failures
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# awk '$4=="FAIL" {n[$3]++} END {
  for(u in n) if(n[u]>m) m=n[u]
  for(u in n) if(n[u]==m) print u,n[u]
}' access.log | sort
user3 15
user5 15
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 4.3–4.4 – Questions 3 and 4 results

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo "Q5: Requests and failures for user3"
Q5: Requests and failures for user3
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# awk '$3=="user3" {
  n++
  if($4=="FAIL") f++
} END {printf "user3: %d requests, %d failed\n",n,f}' access.log
user3: 72 requests, 15 failed
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo "Q6: Last three failures — time and user"
Q6: Last three failures — time and user
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# awk '$4=="FAIL" {print $2,$3}' access.log | tail -n 3
10:10 user0
10:15 user5
10:20 user3
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 4.5–4.6 – Questions 5 and 6 results

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo "Q7: Accounts grouped by login shell"
Q7: Accounts grouped by login shell
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# cut -d: -f7 /etc/passwd | sort | uniq -c | sort -nr
     28 /usr/sbin/nologin
      3 /bin/false
      3 /bin/bash
      1 /bin/sync
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo "Q8: Plain listing line count"
Q8: Plain listing line count
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ls /etc | wc -l
215
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo "Q8: Long listing line count"
Q8: Long listing line count
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ls -l /etc | wc -l
216
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 4.7–4.8 – Questions 7 and 8 results

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# cd /tmp/igor-lab3-play-EN7rTn
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# cp /tmp/igor-os-review-20261109/Lab3-4/linux/oneliners.
sh .
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# cp /tmp/igor-os-review-20261109/Lab3-4/linux/report.sh
.
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# bash oneliners.sh access.log

Q1 failed requests
100

Q2 distinct pages
4 distinct pages
/page0
/page1
/page2
/page3

Q3 requests per page
125 /page0
125 /page1
125 /page2
125 /page3

Q4 most failures
user3 15
user5 15

Q5 user3
user3: 72 requests, 15 failed

Q6 last three failures
10:10 user0
10:15 user5
10:20 user3

Q7 login shells
     28 /usr/sbin/nologin
      3 /bin/false
      3 /bin/bash
      1 /bin/sync

Q8 directory listing line counts
215
216
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 4.9 – Full run of oneliners.sh

Part 5 The report script and its error path

The shebang #!/bin/bash selects Bash when the executable script is launched directly. Before chmod +x, ./report.sh returned permission denied and status 126. The prefix ./ supplies a pathname because the current directory is not searched through PATH here. The error is written to descriptor 2 so stdout remains usable for normal results; exit 1 lets a caller detect failure. The script prints every user from user0 to user6, including zero counts, using its aggregation loop. Optional cron and bonus tasks are not claimed.

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# cat -n report.sh
     1  #!/bin/bash
     2  if [ "$#" -ne 1 ]; then
     3      printf 'usage: %s readable_log\n' "$0" >&2
     4      exit 1
     5  fi
     6  if [ ! -f "$1" ] || [ ! -r "$1" ]; then
     7      printf 'no such file: %s\n' "$1" >&2
     8      exit 1
     9  fi
    10  awk '
    11    { total++ }
    12    $4 == "FAIL" { failed++; by_user[$3]++ }
    13    END {
    14      printf "%d requests, %d failed\n", total, failed
    15      for (i=0; i<7; i++) printf "user%d: %d failures\n", i, by_user["user" i]
    16    }
    17  ' "$1"
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 5.1 – Source of report.sh

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# chmod 644 report.sh
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ./report.sh access.log
-bash: ./report.sh: Permission denied
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo "status before chmod: $?"
status before chmod: 126
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# chmod +x report.sh
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo "status after chmod: $?"
status after chmod: 0
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 5.2 – Successful report and execute permission check

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# ./report.sh no_such_igor_log.txt
no such file: no_such_igor_log.txt
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn# echo "status: $?"
status: 1
root@ubuntu-4gb-hel1-1:/tmp/igor-lab3-play-EN7rTn#
```

Figure 5.3 – Missing file diagnostic and status

Conclusions

Argument expansion belongs to Bash before program execution. Descriptor duplication explains the order-sensitive redirection result, while status-dependent operators make failure handling explicit. The one-liners were verified against fixed input totals and the reusable script was tested on success and missing input. Source files, predictions and raw recordings accompany this report.

MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers Informatics and Microelectronics

Department of Software Engineering and Automation

Report on laboratory work 4

The xv6 shell from the outside

Course Operating Systems

Done by: st. gr. FAF-242, Tacu Igor
Checked by: Reițman Patricia

Chișinău, 2026

Purpose and execution record

All twelve figures are native Terminal screenshots supplied by the author from direct command execution and source inspection on the Ubuntu server on 9 November 2026. The xv6 tests use an isolated source copy and its generated test disk. The directory d already existed during the directory-handling capture; grep READ limits the displayed listing. Bash comparisons use separate temporary directories. Source files and earlier full execution transcripts accompany the screenshots. The submission was prepared with coding assistance; MIT xv6 is identified as upstream in the source provenance file.

Part 1 Sleep and the current interface

The header declares pause rather than sleep. The submitted utility calls pause with validated decimal ticks; sleep 20 returns normally, missing input produces usage, and negative input produces a diagnostic. The Makefile contains the _sleep entry. Forked processes inherit the binary and descriptor context, while pause is the kernel operation that performs the delay.

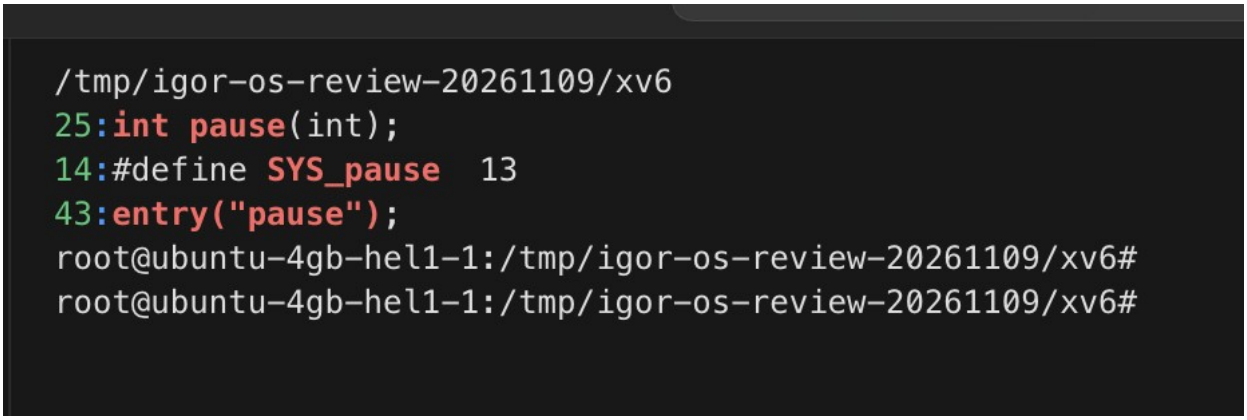


Figure 1.1 – The pause declaration

```
 1  #include "kernel/types.h"
 2  #include "user/user.h"
 3
 4  int
 5  main(int argc, char **argv)
 6  {
 7    if(argc != 2 || argv[1][0] == 0){
 8      fprintf(2, "usage: sleep nonnegative_ticks\n");
 9      exit(1);
10    }
11    int ticks = 0;
12    for(char *p = argv[1]; *p; p++){
13      if(*p < '0' || *p > '9' || ticks > (2147483647 - (*p - '0')) / 10){
14        fprintf(2, "sleep: invalid tick count\n");
15        exit(1);
16      }
17      ticks = ticks * 10 + (*p - '0');
18    }
19    exit(pause(ticks) < 0);
20  }
138:    $U/_sleep\
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6#
```

Figure 1.2 – Sleep source and registration

```
xv6 kernel is booting

init: starting sh
$ sleep 20
$ echo finished
finished
$ sleep
usage: sleep nonnegative_ticks
$ sleep -1
sleep: invalid tick count
$
```

Figure 1.3 – Sleep with valid and invalid arguments

Part 2 Comparing every command in the two shells

The Bash test starts in a fresh directory containing README. Both shells provide execution, descriptor redirection, pipelines, command lists, grouping and background commands. Bash

additionally expands wildcards and variables, removes syntactic quotes, and offers readline history and completion. The table records the actual small-shell behavior rather than assuming it implements Bash syntax.

Command	Bash	xv6	Reason
echo hello > f ; cat < f	hello	hello	descriptor redirection
ls | grep READ	README	README line	pipe
(echo a; echo b) | wc	2 lines 2 words 4 bytes	2 2 4	group and list
echo one ; echo two	one then two	one then two	sequence
echo bg &	bg and prompt	bg and prompt	background
ls *	expands existing names	cannot open *	Bash pathname expansion
echo $HOME	/root	$HOME	Bash variable expansion
echo "a b"	a b	"a b"	Bash quote removal
mkdir d ; cd d ; ls ..	lists parent after changing directory	exec cd failed; lists from original directory	xv6 handles cd only as a whole input line
Up Tab Ctrl+R	history and completion	not implemented as Bash readline features	source and PTY inspection

There is an important distinction in the last directory case. A cd embedded in a semicolon list is passed through the xv6 execution parser and fails as an external command. Issuing cd d as its own input line does change the parent shell. From there a bare ls fails because xv6 has no PATH search, while /ls .. succeeds. For interactive work, history and completion are valuable additions. Basic quote handling would be a localized tokenizer improvement; full pathname expansion would also require directory traversal.

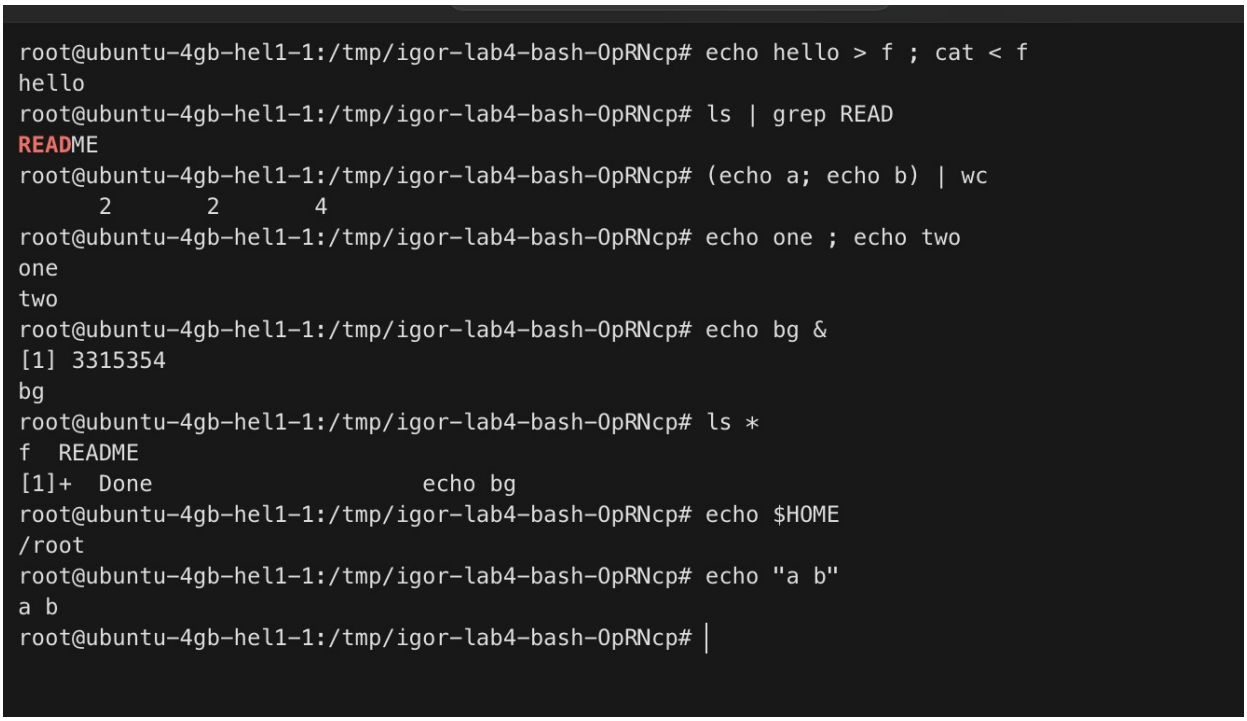


Figure 2.1 – The original comparison commands in Bash

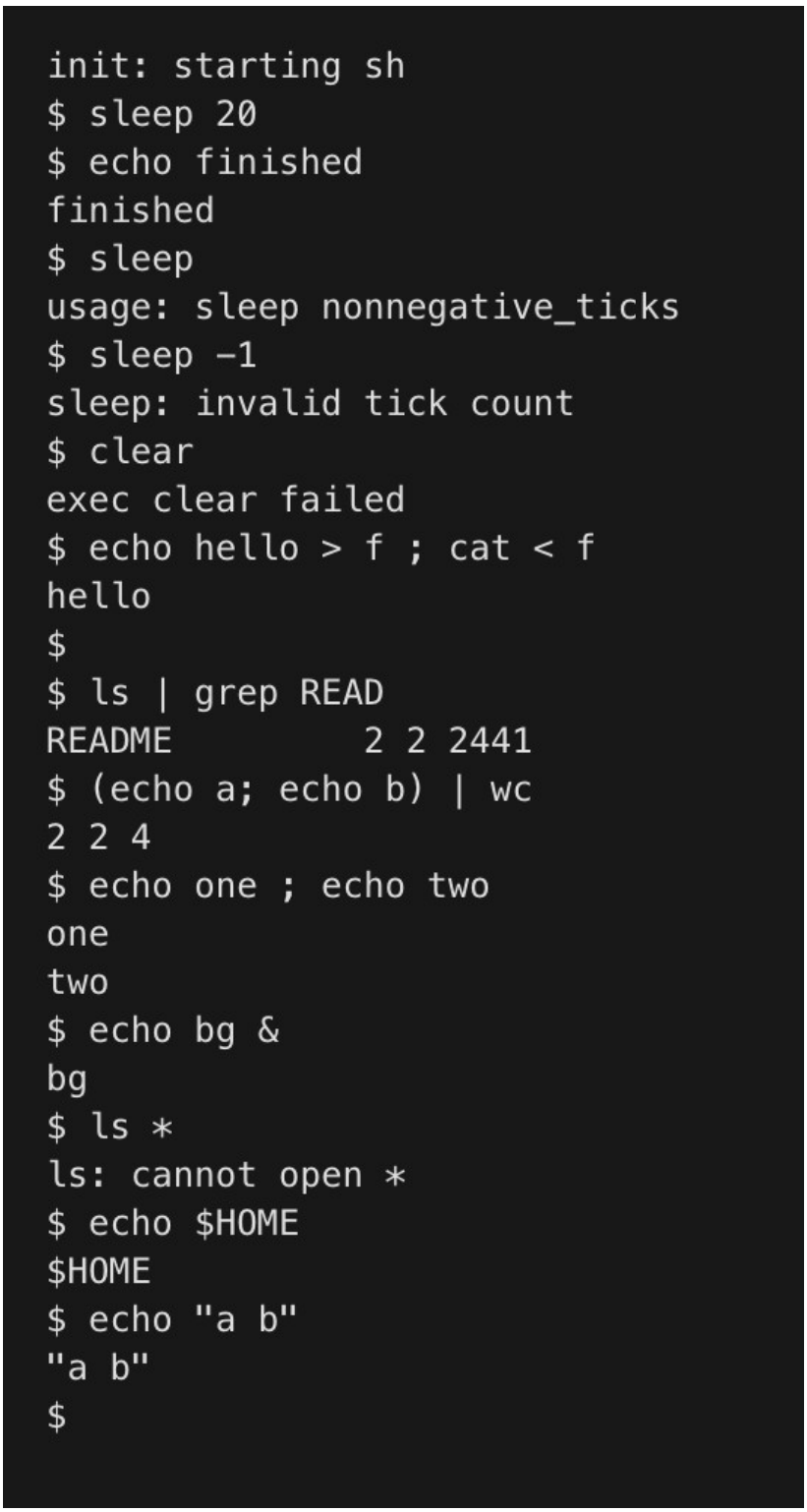


Figure 2.2 – The same comparison commands in xv6



Figure 2.3 – Directory handling and absolute ls (README-filtered listing)

Part 3 Who expands the argument vector

Both inspectors print only argc and argv. The Linux wildcard experiment in this additional test directory had four C files and one quoted x y operand, giving argc 6 including argv zero. Xv6 passed four arguments: args, the literal *.c, and two values containing the double quote marks. The shell, not the inspector or kernel, is responsible for expansion and syntactic quote removal. Repeated unquoted spaces delimit arguments in both shells; they do not become empty arguments.

```
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6# cat -n user/args.c
     1  #include "kernel/types.h"
     2  #include "user/user.h"
     3
     4  int
     5  main(int argc, char **argv)
     6  {
     7    printf("argc=%d\n", argc);
     8    for(int i = 0; i < argc; i++) printf("argv[%d]=[%s]\n", i, argv[i]);
     9    exit(0);
    10  }
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6# grep -n '_args' Makefile
139:    $U/_args\
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6#
```

Figure 3.1 – Args source and UPROGS entry

```
$ args one    two three
argc=4
argv[0]=[args]
argv[1]=[one]
argv[2]=[two]
argv[3]=[three]
$
$ args *.c "x y"
argc=4
argv[0]=[args]
argv[1]=[*.c]
argv[2]=["x]
argv[3]=[y"]
$ args $HOME
argc=2
argv[0]=[args]
argv[1]=[$HOME]
$ args 'single quoted'
argc=3
argv[0]=[args]
argv[1]=['single]
argv[2]=[quoted']
$
```

Figure 3.2 – Argument arrays from xv6

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o# ./args one    two three
argc=4
argv[0]=[./args]
argv[1]=[one]
argv[2]=[two]
argv[3]=[three]
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o# ./args *.c "x y"
argc=6
argv[0]=[./args]
argv[1]=[alpha.c]
argv[2]=[args_linux.c]
argv[3]=[beta.c]
argv[4]=[sample.c]
argv[5]=[x y]
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o# ./args $HOME
argc=2
argv[0]=[./args]
argv[1]=[/root]
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o# ./args '$HOME' "$HOME"
argc=3
argv[0]=[./args]
argv[1]=[$HOME]
argv[2]=[/root]
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o# ./args 'single quoted'
argc=2
argv[0]=[./args]
argv[1]=[single quoted]
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o#
```

Figure 3.3 – Argument arrays from Linux

Part 4 Reading the shell

This version of user/sh.c has 499 lines. EXEC represents a plain command; REDIR is introduced by < or >; PIPE by |; LIST by a semicolon; and BACK by ampersand. Parentheses group a parsed command tree. Cd is recognized in the parent loop before forking and parsing a general command, so its directory change persists. The parser has no Bash parameter-expansion or globbing stage, explaining the literal * and $HOME results. The five command kinds are visible both in their definitions and in runcmd dispatch.

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o# cd /tmp/igor-os-review-20261109/xv6
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6# grep -nE '^#define (EXEC|REDIR|PIPE|LIST|BACK)' u
ser/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6#
```

Figure 4.1 – Defined command kinds

```
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6# grep -nE 'case (EXEC|REDIR|PIPE|LIST|BACK)' user/
sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6# \
```

Figure 4.2 – Dispatch cases

```
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6# sed -n '159,177p' user/sh.c
  // Read and run input commands.
  while (getcmd(buf, sizeof(buf)) >= 0) {
    char *cmd = buf;
    while (*cmd == ' ' || *cmd == '\t')
      cmd++;
    if (*cmd == '\n') // is a blank command
      continue;
    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
      // Chdir must be called by the parent, not the child.
      cmd[strlen(cmd) - 1] = 0; // chop \n
      if (chdir(cmd + 3) < 0)
        fprintf(2, "cannot cd %s\n", cmd + 3);
    } else {
      if (fork1() == 0)
        runcmd(parsecmd(cmd));
      wait(0);
    }
  }
  exit(0);
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6#
```

Figure 4.3 – Parent shell implementation of cd

Conclusions

The two shells share process and descriptor mechanisms but differ before exec in parsing and expansion. The argument inspector isolates those differences. The additional cd-list test also shows why a small shell feature cannot be assumed to compose exactly as in Bash. Both requested programs and the integrated Makefile are submitted with the full xv6 source and raw console evidence.